

PYTHON FOR BEGINNERS

SESSION 01

Variables, Data Types & Print Statements

What is Python?

Python is a high-level, general-purpose programming language. It was created by **Guido van Rossum** and first released in **1991**. The goal behind Python was simple – write code that reads almost like plain English, so developers can focus on solving problems rather than fighting with syntax.

- **Easy to read and write** → minimal boilerplate, clean syntax
- **Interpreted** → code runs line by line, no compilation needed
- **Dynamically typed** → you don't declare variable types manually
- **Hugely versatile** → web, data, AI, automation, all of it

Where is Python Used?

Field	What Python does there
Web Development	Build backend servers and APIs
Data Science	Analyse and visualise data
Machine Learning & AI	Train models, build intelligent systems
Automation & Scripting	Automate repetitive tasks
Data Engineering	Build data pipelines and ETL workflows
Cybersecurity	Write scripts for penetration testing

Companies Using Python

Pretty much every major tech company uses Python in some capacity:

- **Google** → uses Python heavily in internal tools, YouTube backend, and AI research.
- **Instagram** → one of the largest Django (Python) applications in the world.
- **Netflix** → uses Python for data science, recommendations, and automation.
- **Uber** → Python powers their data pipelines and backend microservices.
- **Dropbox** → their desktop client was originally built entirely in Python.
- **NASA** → uses Python for scientific computing and data analysis.

Note:

In **India** specifically, companies like Flipkart, Swiggy, Zerodha, and Razorpay all use Python heavily in their backend and data teams so placement-wise, Python is one of the most in-demand skills right now.

What is a Variable?

A variable is a name that stores a value in memory. Think of it as a labelled box you give the box a name, put something inside, and refer to it whenever you need it.

```
name = "Rahul"  
age  = 21  
gpa  = 8.5  
is_student = True  
  
print(name)  
print(age)
```

Note: Python does not require you to declare the type of a variable. It figures it out automatically based on the value you assign.

Variable Naming Rules

- ✓ Must start with a letter or underscore (_)
- ✓ Can contain letters, digits, and underscores
- ✗ Cannot start with a digit
- ✗ Cannot use Python keywords (if, for, while...)
- ✓ Names are case-sensitive → age ≠ Age ≠ AGE

```
# Valid
student_name = "Priya"
_count = 10
totalMarks2 = 95

# Invalid
2marks = 95      # starts with digit
my-name = "Raj"  # hyphens not allowed
```

Data Types in Python

Every value in Python has a type. Python decides the type automatically you just assign the value.

Type	Keyword	Example	What it stores
Integer	int	42	Whole numbers
Float	float	3.14	Decimal numbers
String	str	"Hello"	Text
Boolean	bool	True / False	Yes or No values

Coming Later: Python also has *complex* (complex numbers) and *None* (NoneType). We will cover these in later sessions.

The `print()` Function

The *print()* function outputs values to the screen. You can pass one value or multiple values separated by commas.

```
# Single value
print("Hello, World!")

# Multiple values
name = "Riya"
age = 20
print("Name:", name, "| Age:", age)
# Output: Name: Riya | Age: 20

# Printing numbers and expressions
print(10 + 5)      # 15
print(type(3.14))  # <class 'float'>
```

print(): sep and end

By default, print puts a space between multiple values and a newline at the end. You can change both using sep and end.

```
# sep: changes the separator between values
print("2024", "01", "15", sep="-")      # 2024-01-15
print("a", "b", "c", sep=" | ")        # a | b | c

# end: changes what is printed at the very end
print("Loading", end="...")
print("Done")                          # Loading...Done

# Combining both
print("10", "20", "30", sep=", ", end="!\n")  # 10, 20, 30!
```

print(): Using F-Strings

An *f-string* lets you embed variable values directly inside a string. Put an **f** before the opening quote and write variables inside { }.

```
name    = "Karan"
age     = 22
course  = "Python"

# Without f-string (messy)
print("Hello, " + name + ". You are " + str(age) + " years old.")

# With f-string (clean and readable)
print(f"Hello, {name}. You are {age} years old.")
print(f>Welcome to the {course} course!")

# Simple math inside {}
price = 499
qty   = 3
print(f>Total: {price * qty}")
```

Type Conversion: Implicit vs Explicit

Implicit – Python does it automatically

```
x = 5          # int
y = 2.0        # float
z = x + y      # Python converts x to float automatically
print(z)       # 7.0
print(type(z)) # <class 'float'>
```

Explicit – You do it manually

```
num_str      = "42"
num_int      = int(num_str)    # "42" → 42
num_float    = float(num_str)  # "42" → 42.0
back_to_str  = str(100)        # 100 → "100"
```

Taking User Input with input()

The **input()** function pauses the program and waits for the user to type something. Whatever the user types is always returned as a string.

```
name = input("Enter your name: ")    # user types: Rahul
print(f"Hello, {name}!")
```